

Lab 4 - SMART on FHIR

- Due Monday by 8am
- Points 6
- Submitting an external tool
- Available Jan 12 at 8am - Feb 23 at 8am

Lab 4 - SMART on FHIR

Before attempting this lab you must watch the FHIR Technical Series: SMART on FHIR lecture.

Overview

For this lab, you will deploy a SMART on FHIR app leveraging the SMART on FHIR sandbox server. SMART on FHIR is a powerful approach to using FHIR to build out numerous, smaller applications that work along with existing EHR systems.

In this exercise, you are tasked to work on several features as part of a healthcare application that will manage medications within an acute care hospital setting.

The code base is written in TypeScript using the Angular framework and will be deployed locally using the Angular development server. (No Angular knowledge will be required.) All of your work will be in the `student-code.service.ts` file, which is what will be submitted to Gradescope.

Learning Objectives

- Learn how FHIR APIs are used in modern healthcare apps.
- Learn how to use SMART on FHIR to build secure healthcare apps that run on most SMART on FHIR EHR environments.
- Learn how to use the Client.js SMART on FHIR library to perform CRUD operations.

Tools Used

- [Lab 4 - SMART on FHIR Student Starter Code](https://github.gatech.edu/cs6440-students/Lab-4-SMART-on-FHIR)  (<https://github.gatech.edu/cs6440-students/Lab-4-SMART-on-FHIR>)
- [SMART on FHIR](https://docs.smarthealthit.org/)  (<https://docs.smarthealthit.org/>)
- [SMART Health IT FHIR Sandbox](https://launch.smarthealthit.org/)  (<https://launch.smarthealthit.org/>)
- Client.js (SMART on FHIR JavaScript Library)

- Node.js version 18
- Angular
- **Node Version Manager (NVM)** [↗\(https://github.com/nvm-sh/nvm\)](https://github.com/nvm-sh/nvm) ***RECOMMENDED***

Additional Resources

- **Client.js Documentation** [↗\(http://docs.smarthealthit.org/client-js/client.html\)](http://docs.smarthealthit.org/client-js/client.html)
- FHIR Specification (**Observation Resource** [↗\(http://hl7.org/fhir/R4/observation.html\)](http://hl7.org/fhir/R4/observation.html) | **MedicationRequest Resource** [↗\(http://hl7.org/fhir/R4/medicationrequest.html\)](http://hl7.org/fhir/R4/medicationrequest.html))
- **Angular docs** [↗\(https://angular.io/docs\)](https://angular.io/docs)

Getting Started

Setting Up The Starter Code

1. **(OPTIONAL, BUT RECOMMENDED)** Download and install Node Version Manager. This is a tool that will allow you to more easily maintain different versions of Node and streamline the installation process at the same time. To install NVM, please follow the instructions in their GitHub README at <https://github.com/nvm-sh/nvm#installing-and-updating>. [↗\(https://github.com/nvm-sh/nvm#installing-and-updating\)](https://github.com/nvm-sh/nvm#installing-and-updating)
2. Install Node.js version 18 from <https://nodejs.org/en/download> [↗\(https://nodejs.org/en/download\)](https://nodejs.org/en/download).
 1. If not using NVM, follow the instructions in the Node.js documentation for your device.
 2. If using NVM, use the following steps:
 1. `nvm install node` if you have never used NVM before, this should download the latest version and ensure NVM is aware of all versions.
 2. `nvm list`, note the label for version 18. It should be "lts/hydrogen".
 3. `nvm install lts/hydrogen` to install version 18.
 4. `nvm use lts/hydrogen` to change your local environment to the proper version.
 5. If you wish to set your default version to 18, run `nvm alias default lts/hydrogen` otherwise you will have to run the previous command each time you reopen your environment.
3. Clone or download the starter code repo from here: <https://github.gatech.edu/cs6440-students/Lab-4-SMART-on-FHIR> [↗\(https://github.gatech.edu/cs6440-students/Lab-4-SMART-on-FHIR\)](https://github.gatech.edu/cs6440-students/Lab-4-SMART-on-FHIR)
4. In your terminal, navigate to the working directory of the repo. Type the command `npm install`. This will install the needed node modules that are defined in the package.json file.
5. Install the Angular CLI using `npm install -g @angular/cli`. (Note: -g is the global flag to install it beyond the current project workspace.)

6. Once all the node modules are downloaded in the command line type `ng serve`. This will deploy the front end code <http://localhost:4200>  (<http://localhost:4200/>)

Setting the SMART on FHIR Sandbox Launcher

These steps outline how to setup the SMART on FHIR Sandbox. This will simulate launching a SMART app with patient context, OAuth, and a FHIR server.

1. Go to the SMART on FHIR Sandbox Launcher site <https://launch.smarthealthit.org/>  (<https://launch.smarthealthit.org/>)
2. Ensure that you have selected FHIR R4 and a launch type of Provider EHR launch.
3. Uncheck "Simulate launch within the EHR UI" if it is selected, you should not use this. (It is buggy.)
4. Set the App Launch URL to: <http://localhost:4200/launch/>
5. Click on launch app to deploy the app and be sent into the simulated authorization workflow.

Note: You can provide the context of a patient on the launch screen if you would like, though it is not required as you will be given an option to select a patient while loading.

Exercises

All of your code should be written in the **student-code.service.ts** file.

Your primary task will be to edit the starter SMART on FHIR starter template code to add additional features for the app. You are going to add the ability to search for the patient's MedicationRequest resources, update the status of a given MedicationRequest (for example, changing one from "active" to "stopped"), and to order a new medication for the patient (create a MedicationRequest).

We have provided the code on how the observations are pulled. This will be helpful to see how the SmartJS Client works for a simple query, but you will need to consult the linked documentation and understand what you are requesting through the FHIR API for additional details.

In addition to the 3 code based exercises, you will complete a small function that returns a string containing what your SMART on FHIR scopes would be in a non-simulated environment. You are given the following scopes in the project's environment.ts file:

```
launch profile openid online_access patient/Patient.read
```

There are two additional scopes you must add to this. Scopes should be kept to a minimum requirement and follow SMART on FHIR 1.0 (which is still the most common implementation) guidelines here: <http://hl7.org/fhir/smart-app-launch/1.0.0/scopes-and-launch-context/index.html>  (<http://hl7.org/fhir/smart-app-launch/1.0.0/scopes-and-launch-context/index.html>).

For example, if the application were to make search requests for a patient's encounters, you would add a scope for "patient/Encounter.read". If, and **only** if, there was an expectation the application would write to the server for a given resource, you would use a wildcard, such as "patient/Encounter.*".

For each exercise, a "client" object will be given to the function. You do not have to do anything to provide this, it is created for you in advance. This is the SMART on FHIR Client object you will use to perform your queries.

For every return from a function, you will need to wrap your client call in the "from<any[]>()" structure as in the example code. This converts the basic JavaScript Promise object (used for things such as HTTP calls) into an RXJS Observable. You do not need to be concerned with the workings of Observables, they are standard convention in Angular and so we could not entirely obscure them here. As long as you use the approach shown, your code will be fine.

Note that for the placeholders, another wrapper is use "of()" which performs a similar function to "from()", though it is used to create an Observable of simple data rather than convert a Promise to an Observable.

If you would like to read more, you can do so at the following links:

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise  (https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Promise)
- <https://rxjs.dev/guide/observable>  (<https://rxjs.dev/guide/observable>)
- <https://rxjs.dev/api/index/function/from>  (<https://rxjs.dev/api/index/function/from>)
- <https://rxjs.dev/api/index/function/of>  (<https://rxjs.dev/api/index/function/of>)

Exercise 1 - Search Medication Requests

The first feature is to list all of the MedicationRequest resources for the current patient as a **list** (not a searchset Bundle!). For this exercise, you should follow the pattern established in the example for searchObservations, but adjust as needed.

Additional Requirements and Constraints

- You must include the `{flat: true}` parameter so that the client provides the results as a list instead of the bundle.
- You must not include any additional parameters (even if it is in the example) that will alter the contents of the return from the most simple form of the request being asked for.

Exercise 2 - Update MedicationRequest

The second task is to update a MedicationRequest resource's status field in the function `exercise_2_updateMedicationRequest`. The MedicationRequest resource being updated and the new status value is passed when the field is modified through the application's UI.

You must update the field in the object, and then make a call to the server with the updated resource.

Additional Requirements and Constraints

- You should consult the Client documentation given to you in the resources section of this page for the appropriate call to make.
- You must not use the request method as with the search, you must use the appropriate method wrapper. (Read the documentation carefully.)

Exercise 3 - Create MedicationRequest

The third task is to create the functionality to order a create a new MedicationRequest resource using the function `exercise_3_createMedicationRequest`.

You should build a new resource as JSON. (Consult the FHIR documentation for examples.) Include only the following required fields:

- **resourceType**,
- **status** (should be set to "draft")
- **intent** (should be set to "order")
- **subject** (should be a reference to the current patient)
- **medicationCodeableConcept.text** (should be set to the string passed in by the form)
 - *DO NOT SET ANY OTHER VALUES IN THIS FIELD, ONLY TEXT!*

Additional Requirements and Constraints

- You should consult the Client documentation given to you in the resources section of this page for the appropriate call to make.
- You must not use the request method as with the search, you must use the appropriate method wrapper. (Read the documentation carefully.)
- You must include every field listed above.
- You must not add additional fields not listed above.
- (Hint) Pay attention to the difference between nested JSON and an object, use debuggers or ``console.log()`` statements to spot discrepancies in what you are passing the method you are using.

Submission

Submit the **student-code.service.ts** file to Gradescope.